OPEN PLATFORM FIRMWARE (OPF)

Advancing Flash-less Boot in OCP Systems: A Streaming Firmware Architecture for Secure and Scalable Infrastructure

Munir Ahmad

Chief Compute Architect, Lattice Semiconductor

Alex Cervantes

Cloud and AI Systems Architect, Intel

OCP PLATINUM Member

The Problem: Why Move Beyond Local Flash?

Supply Chain Attack Surface

SPI NOR flash is physically accessible; tamper is hardware-invisible. Local flash = persistent attack vector in every server chassis.

Verification Granularity

Traditional flash reads are unverified per-page. NIST SP 800-193 requires Protect-Detect-Recover, hard to achieve at boot-time with local NOR.

Firmware Update Complexity

Updating 10,000+ nodes requires in-band flash erase/write cycles. Failed updates brick devices. No atomic rollback.

No Central Policy Enforcement

Firmware versions diverge silently across fleets. Zero-touch, policy-driven firmware management requires eliminating local state.

Streaming Boot vs Flash-less Boot

Streaming boot and Flash-less boot are closely related but are not exactly same

Streaming boot

BMC / Host (Streams firmware image)-→ RoT / FPGA / → Loads firmware into SRAM → Executes code

- A system architecture where the device does not contain local flash memory to store firmware permanently.
- Describes the storage model.
- Eliminates onboard SPI flash/NAND.

- No persistent firmware storage on the device
- Firmware images are stored elsewhere
  - BMC
  - Host CPU
  - Management Controller
  - Network server
- Device boots into a small immutable ROM.
- ROM requests or receives firmware from an external source

Flash-less boot

ROM → Receive image in chunks → Verify image → Store in SRAM → Execute

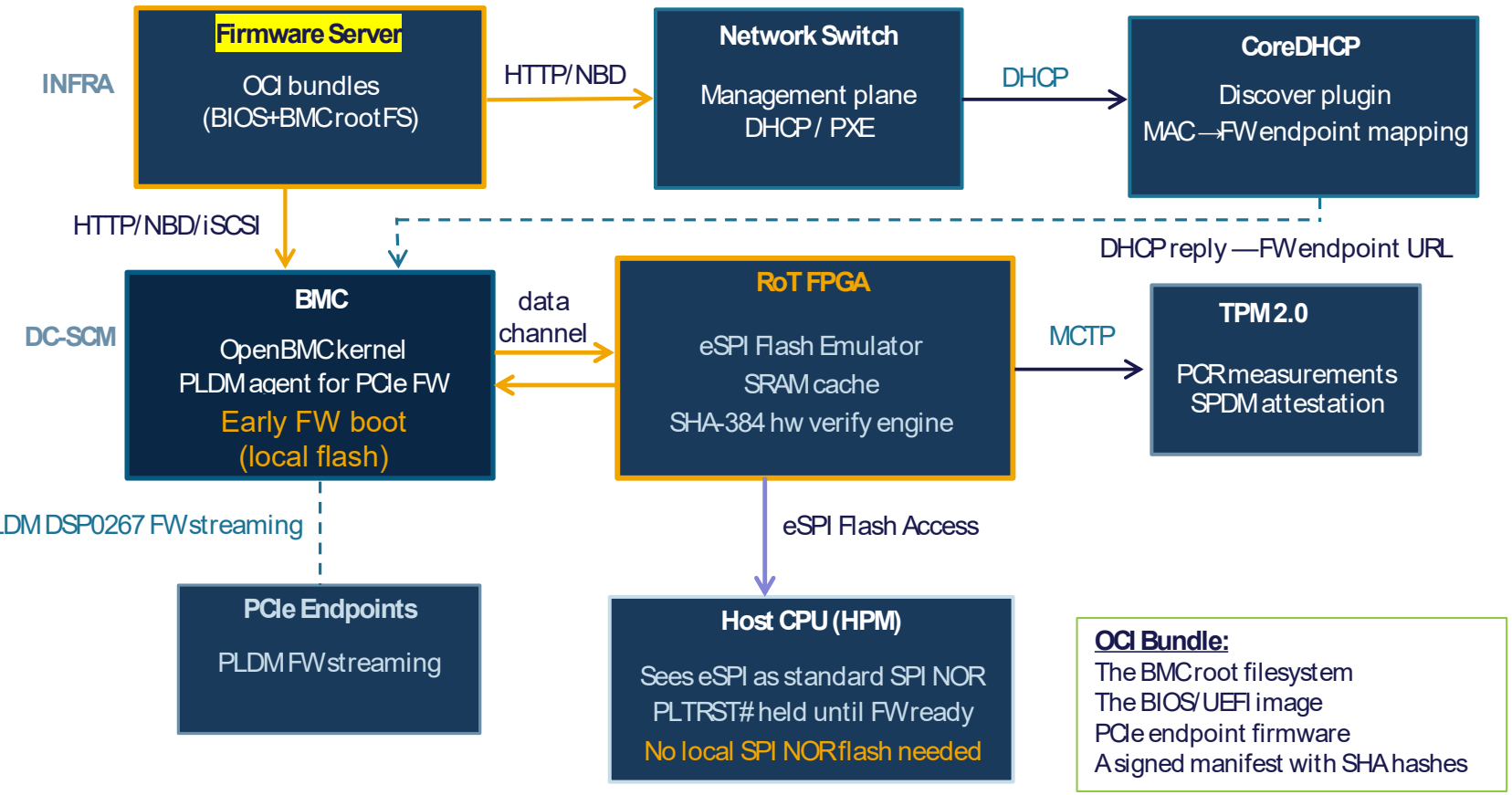
- A boot mechanism where firmware is sent to the device during boot time from an external source.
- Describes the firmware delivery method.
- Transfers firmware over a communication interface (I3C, MCTP, PCIe, USB, AXI, etc.).

- OCP's streaming boot initiative uses a recovery-style protocol where a management controller streams the firmware image to the Root of Trust. This enables flash-less designs and secure recovery
- The image arrives over interfaces such as
  - I3C
  - MCTP
  - PCIe DOE
  - USB
  - Ethernet

Architectural Distinctions of Two Complementary Approaches

	Flash-less boot	Streaming boot
Locus of Control	Flash-less: BMC / Software Layer, BMC orchestrates all firmware delivery; single trust anchor at software layer	Streaming: Hardware / Protocol Layer : FPGA or RoT ASIC arbitrates every firmware byte before it reaches the CPU
Verification Granularity	Flash-less: Complete image verified before execution , one hash per firmware bundle	Streaming: Per-page (4 KB–64 KB chunk) SHA-256 in real time - detects tampering earlier, no full pre-download required
Transport Dependency	Flash-less: Fundamentally network-dependent — not suitable for isolated environments without management network	Streaming: Operates over local hardware buses (eSPI, PCIe) - streaming controller co-located on DC-SCM eliminates external dependency
Complementarity	Flash-less: Flash-less BMC (network boot of OpenBMC)	Streaming: Streaming BIOS delivery (FPGA eSPI emulation) - combining both gives maximum security + flexibility

Flash-less Boot – End to end block diagram

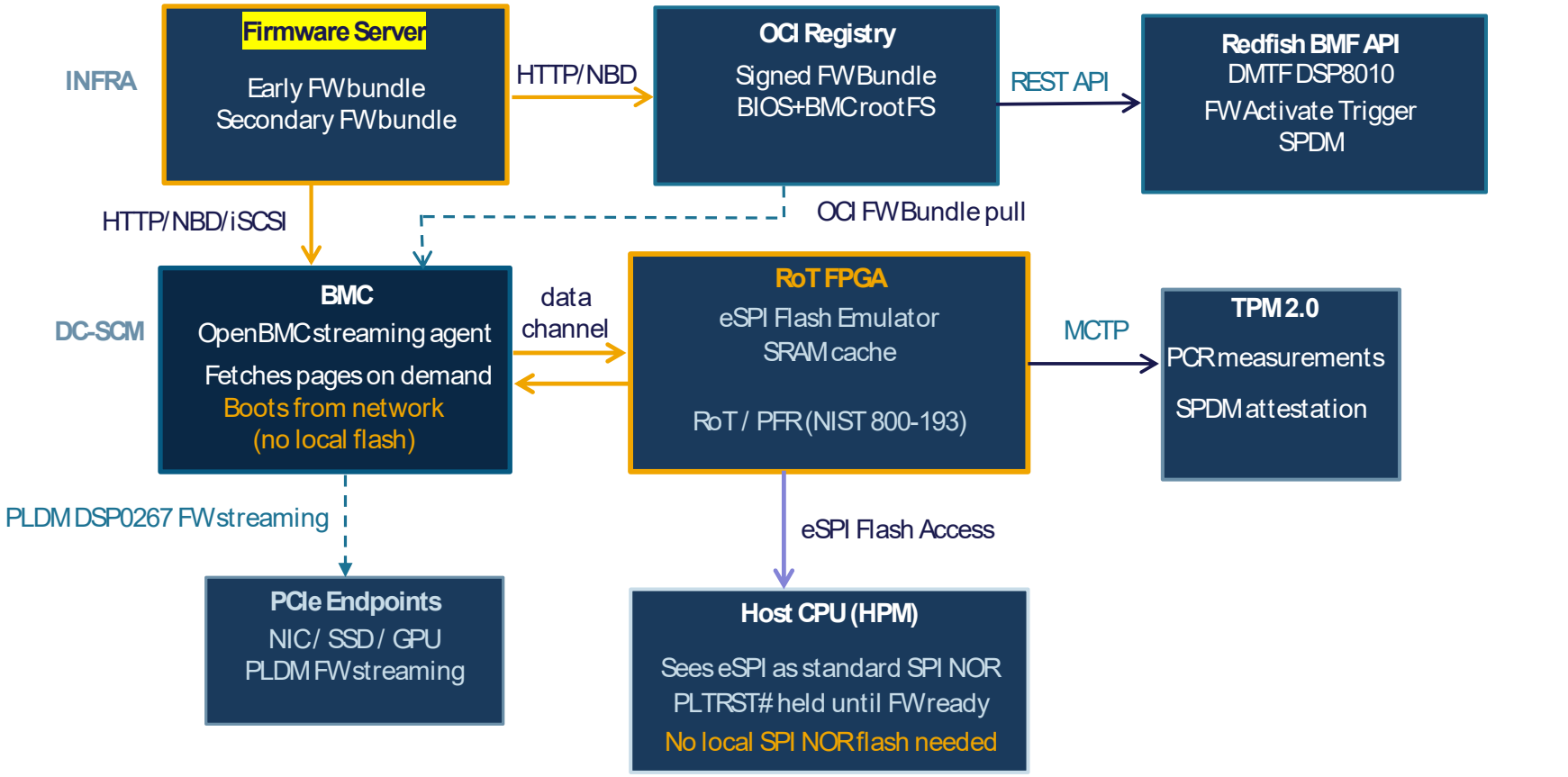


Simple timing sequence:

t=0	Power applied, FPGA configures from OTP, CPU in reset
t=50ms	MC powers on, issues DHCP
t ~ 1s	BMC boots into Linux.
t ~ 1-2s	BMC fetches BIOS Early FW and streams it to FPGA.
t ~ 2s	FPGA SRAM fully loaded and verified. CPU fetches from it.
t ~ 2-6s	CPU runs Early FW → DRAM training
t ~ 6s	DRAM completed. Early FW / Secondary FW boundary crossed.
t ~ 6-20s	FPGA in sliding-window SRAM mode. CPU runs UEFI DXE
t > 20s	FPGA sends firmware measurement log to TPM 2.0 via MCTP, OS bootloader starts.

Streaming Boot – End to end block diagram

Similar flow as before
ROM → Receive image in chunks → Verify image → Store in SRAM → Execute



Early FW vs. Secondary FW: The Streaming Challenge

Early FW and Secondary FW for CPU FW phase exist regardless of how FW is delivered

	Early FW refers to the first code the host CPU executes after reset, covering the SEC phase, Cache-as-RAM initialization, trusted execution environment setup, and the Memory Reference Code that trains DRAM.	Secondary FW is the firmware the CPU executes after DRAM training completes, everything from the end of the Memory Reference Code through to OS handoff. It covers the UEFI DXE phase, the BDS phase, option ROM execution for PCIe devices, and the OS Loader
Phase	SEC / PSP early boot until DRAM available	All firmware after DRAM training completes
Size	≤ 4 MB (FSP-T, AMD early AGESA)	32 MB – 256 MB (UEFI DXE, Option ROMs, LinuxBoot)
DRAM Available	No , CPU uses Cache-as-RAM (64–512 KB)	Yes , DRAM available as large demand-page buffer
Streaming Model	Pre-fetch into FPGA SRAM before PLTRST# (hard real-time, CPU stalls on miss)	Sliding SRAM window + DRAM demand-page (soft real-time, DRAM absorbs latency)
Verification	Full image hash (SHA-384) before CPU release	Per-page (4 KB) SHA-256 from manifest entries
Failure Impact	CPU stall / platform error must be prevented	FPGA WAIT# stretch; retry up to N times; then OCP Recovery

Early FW: Why It Is the Hardest Streaming Problem

Three interconnected constraints make Early FW streaming fundamentally different from any other firmware delivery problem.

1 Zero Latency Tolerance at the Reset Vector

When PLTRST# is deasserted, the x86 CPU immediately fetches from 0xFFFFFFF0. If the FPGA flash emulator cannot respond within the eSPI transaction window, the CPU stalls. Beyond the platform timeout (typically <500 µs for early SEC code), the CPU logs a fatal platform error and the board hangs , unrecoverably without a full power cycle.

2 No DRAM Buffer — FPGA SRAM Is the Only Option

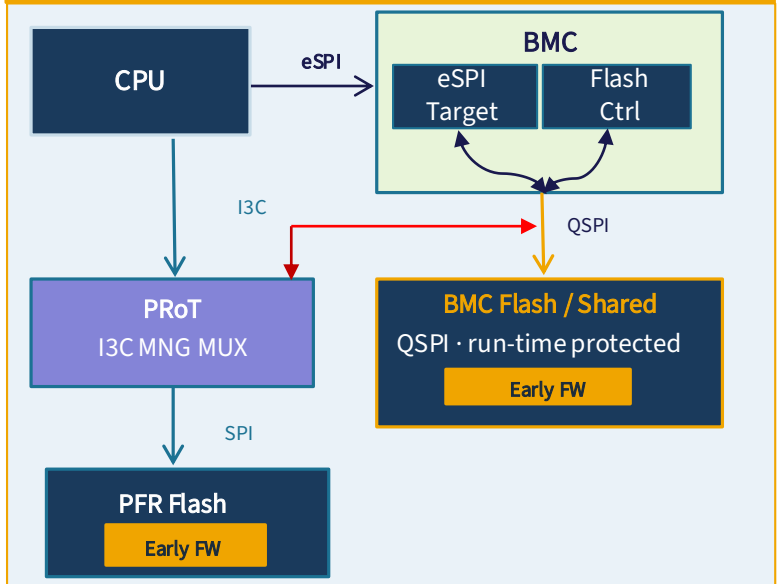
Post-memory streaming can use DRAM as a gigabyte-scale demand-page cache. During Early FW, DRAM does not exist — DDR channels are untrained. The only available buffer is the FPGA internal SRAM, which is small (1–4 MB on MachXO5D). This forces the choice: pre-load the complete Early FW image before releasing the CPU or implement just-in-time streaming with tight per-page delivery guarantees.

3 Sequential, Predictable Access Pattern — an Advantage

Early FW executes almost entirely linearly , from 0xFFFFFFF0 downward through the SEC/PSP code, then a linear CAR setup sequence. Unlike DXE which jumps between drivers, Early FW is predictable. The FPGA can prefetch the next 4 KB page while the CPU executes the current one, hiding network latency behind CPU execution time. This predictability is what makes the pre-fetch strategy viable.

Local Early FW Depository - Three Options Compared

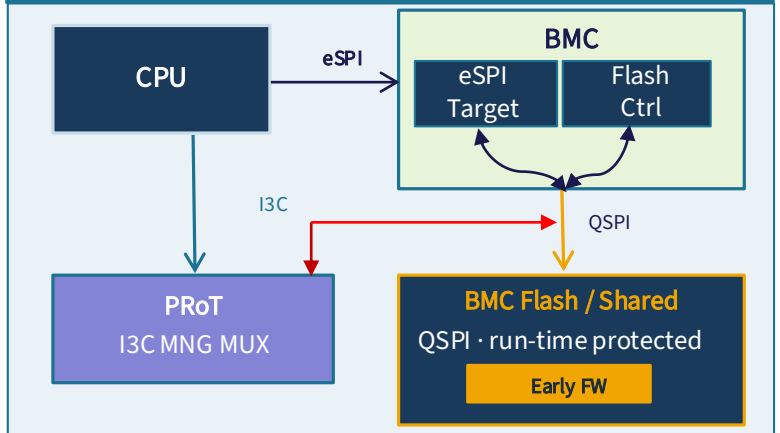
Option 1: BMC SPI Flash + PRoT/PFR Flash



Full redundancy — dual secure copy

Primary: BMC SPI Flash (QSPI)
Backup: PRoT SPI Flash (PFR Flash)
Recovery: I3C MNG MUX arbitration
Best for: highest resilience builds
BOM: medium (extra PFR Flash)

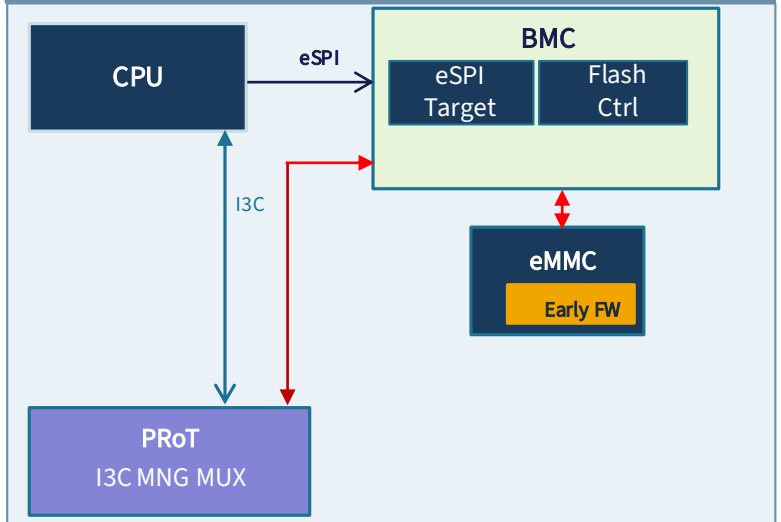
Option 2: BMC SPI Flash + PRoT Internal



Lower BOM · limited capacity

No external PFR Flash device needed
Recovery: PRoT internal flash copy
Capacity: limited by PRoT die size
Typical: Early FW image ≤ 4 MB
eFlash stores manifest
Best for: cost-optimized deployments)

Option 3: eMMC Storage



High capacity · OTA-friendly

Multi-version FW storage (A/B/recovery)
Latency: higher than SPI Flash
Requires eMMC driver in BMC
Best for: OTA update workflows
BOM: higher (eMMC chip + driver stack
Best for: cost-optimized deployments)

Conclusion

Flash-less + Streaming Boot: The Path to Secure, Zero-Touch OCP Infrastructure

- Flash-less boot (HPE Gen11, OVHcloud) validates the production model, fleet-wide in minutes, not hours
- Streaming boot (NVIDIA BlueField, Azure) validates the hardware-protocol model, per-byte RoT verification
- DC-SCM 2.1 + RoT is the enabling platform for combined CPU BIOS streaming , the open engineering gap
- NIST 800-193 + SPDM + PLDM + OCP Recovery v1.1 form the non-negotiable security foundation

Call to Action

- Join the OCP Streaming Boot Initiative
  - OCP Flash-less Boot Specification
    - youtube.com/watch?v=h2DdkbJQARs&t=28
  - OCP DC-SCM 2.1 Hardware Specification
    - https://drive.google.com/file/d/1-SdSQvSWy5pNN_kBiyztblxE4jdyUe9W/view
  - OCP Recovery Spec v1.1:
    - https://docs.google.com/document/d/1Ge_w9i5A6YKG-7nlTp--JhZf6By7I9oB3oW_2_i7JbE/edit?tab=t.0#heading=h.p90lab1o25rh
- Further work
  - Adopt these boot methodologies over next generation of DC-SCM interfaces.
- Discuss
  - What is the biggest barrier to adopting flash-less boot in current platform generation?
  - How does we can handle Early FW recovery today — and what gaps remain under NIST 800-193?
- Other documents
  - Security Protocol & Data Models: DMTF SPDM DSP0274 v1.3
  - Management Component Transport: DMTF MCTP DSP0236

Open Discussion

